

HW6

October 30, 2025

1 Data Loading and Preprocessing

1.1 Load and Explore the Dataset

```
[2]: # a) Load the Amazon baby product reviews dataset using pandas:
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, \
    confusion_matrix

data = pd.read_csv(
    "../doc/GTSC2143-Lecture\
    6_analyzing-product-sentiment-assignment_amazon_baby.csv", index_col=0)

# b) Check basic information:
# - Dataset shape
# - Column names
# - Check any missing value
# - Drop records with missing value

print("Dataset shape:", data.shape)
print("Column names:", data.columns.tolist())
print("Missing values:\n", data.isnull().sum())
data = data.dropna()
print("Shape after dropping missing values:", data.shape)
```

```
Dataset shape: (1000, 3)
Column names: ['name', 'review', 'rating']
Missing values:
name      0
review    3
rating    0
dtype: int64
Shape after dropping missing values: (997, 3)
```

1.2 Create Sentiment Labels

```
[3]: # a) Create a new column called 'positive' where the value is 1 if the rating
      ↪ is greater than 3, and 0 otherwise
data['positive'] = np.where(data['rating'] > 3, 1, 0)

# b) Display the distribution of sentiment labels
print("Sentiment label distribution:")
print(data['positive'].value_counts(normalize=True))
```

Sentiment label distribution:

positive

1 0.715145

0 0.284855

Name: proportion, dtype: float64

2 Data Splitting and Text Processing

2.1 Train/Test Split

```
[4]: # a) Split the data into training (80%) and testing (20%) sets using
      ↪ random_state=42
X_train, X_test, y_train, y_test = train_test_split(
    data['review'], data['positive'], test_size=0.2, random_state=42
)

# b) Display the shapes of training and testing sets
print("Training set shape:", X_train.shape)
print("Test set shape:", X_test.shape)
```

Training set shape: (797,)

Test set shape: (200,)

2.2 Convert Text to Features

```
[5]: # a) Use CountVectorizer to convert review text into word count features
# b) Set max_features=1000 to limit vocabulary size
vectorizer = CountVectorizer(max_features=1000)

# c) Fit the vectorizer on training data and transform both training and test
      ↪ texts
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

# d) Display the shape of the feature matrices
print("\nFeature matrix shapes:")
print("Training:", X_train_vec.shape)
print("Testing:", X_test_vec.shape)
```

Feature matrix shapes:

Training: (797, 1000)

Testing: (200, 1000)

- e) Analysis: Write 2-3 sentences explaining how text becomes numerical features.

Text data cannot be directly used in ML models, so we use CountVectorizer to convert each review into a bag-of-words representation.

Each word becomes a feature, and the value is the count of that word in the review.

This way, the text is turned into numerical data suitable for logistic regression.

3 Model Training and Evaluation

3.1 Logistic Regression Model

```
[6]: # a) Train a logistic regression classifier using the word count features
# b) Use random_state=42 for reproducible results
model = LogisticRegression(random_state=42)
model.fit(X_train_vec, y_train)
```

```
[6]: LogisticRegression(random_state=42)
```

3.2 Evaluate the Model

```
[12]: # a) Make predictions on the test set
y_pred = model.predict(X_test_vec)

# b) Calculate and display:
# - Accuracy score
# - Classification report
# - Confusion matrix
print("Accuracy:\n", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Accuracy:

0.795

Classification Report:

	precision	recall	f1-score	support
0	0.71	0.56	0.63	62
1	0.82	0.90	0.86	138
accuracy			0.80	200
macro avg	0.77	0.73	0.74	200
weighted avg	0.79	0.80	0.79	200

Confusion Matrix:

```
[[ 35  27]
 [ 14 124]]
```

c) Analysis: Write 2-3 sentences interpreting the model's performance.

The logistic regression model achieved an accuracy of **79.5%**, which indicates fairly strong performance for a simple text-based classifier.

The model performs better at identifying **positive reviews** (**recall = 0.90**) than **negative reviews** (**recall = 0.56**), suggesting it is slightly biased toward the majority class (positive sentiment).

Overall, the classifier effectively captures general sentiment trends but could be improved by balancing the dataset or using more advanced text representations such as TF-IDF or word embeddings.

3.3 Feature Analysis

```
[9]: feature_names = np.array(vectorizer.get_feature_names_out())
    coefficients = model.coef_[0]

    # a) Display the top 10 most positive words (highest coefficients)
    top_positive = feature_names[np.argsort(coefficients)[-10:]]
    print("Top 10 Positive Words:", top_positive)

    # b) Display the top 10 most negative words (lowest coefficients)
    top_negative = feature_names[np.argsort(coefficients)[:10]]
    print("Top 10 Negative Words:", top_negative)
```

```
Top 10 Positive Words: ['she' 'has' 'special' 'friend' 'highly' 'once' 'easy'
'without' 'glad'
'love']
```

```
Top 10 Negative Words: ['waste' 'however' 'smells' 'thermometer' 'thought'
'disappointed'
'seemed' 'tried' 'changing' 'maybe']
```

c) Analysis: Write 2-3 sentences about which words drive sentiment predictions.

Words like “love,” “glad,” “easy,” “special,” and “highly” strongly correlate with positive reviews, reflecting customer satisfaction.

Negative sentiment is driven by words like “waste,” “disappointed,” “smells,” and “tried,” which express frustration or poor product quality.